UM6P | College of Computing

Machine Learning for Shock-Capturing Numerical Schemes

Learning reconstruction decisions for 1D hyperbolic conservation laws

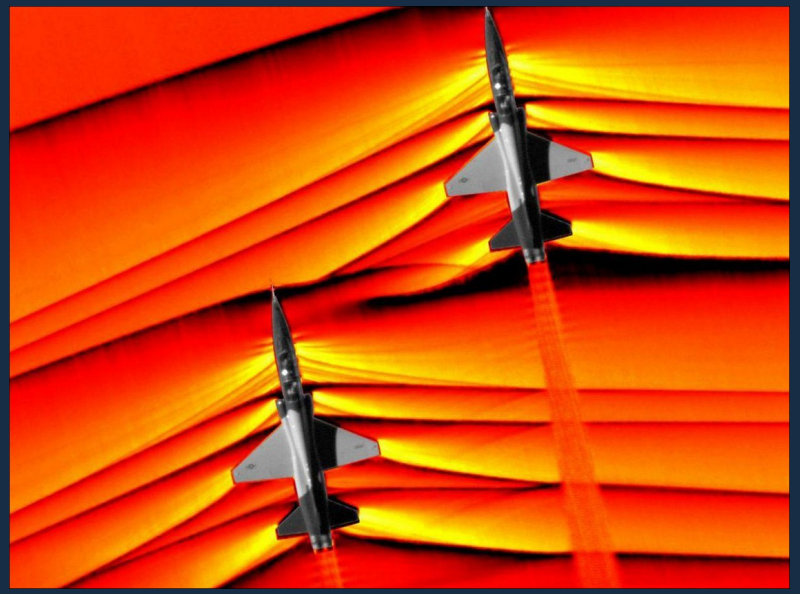
WENO-NN / NN-TENO5 / DIFFERENTIABLE CFD

Yasser Baouzil

Supervisor: Radouan Boukharfane

UM6P · College of Computing

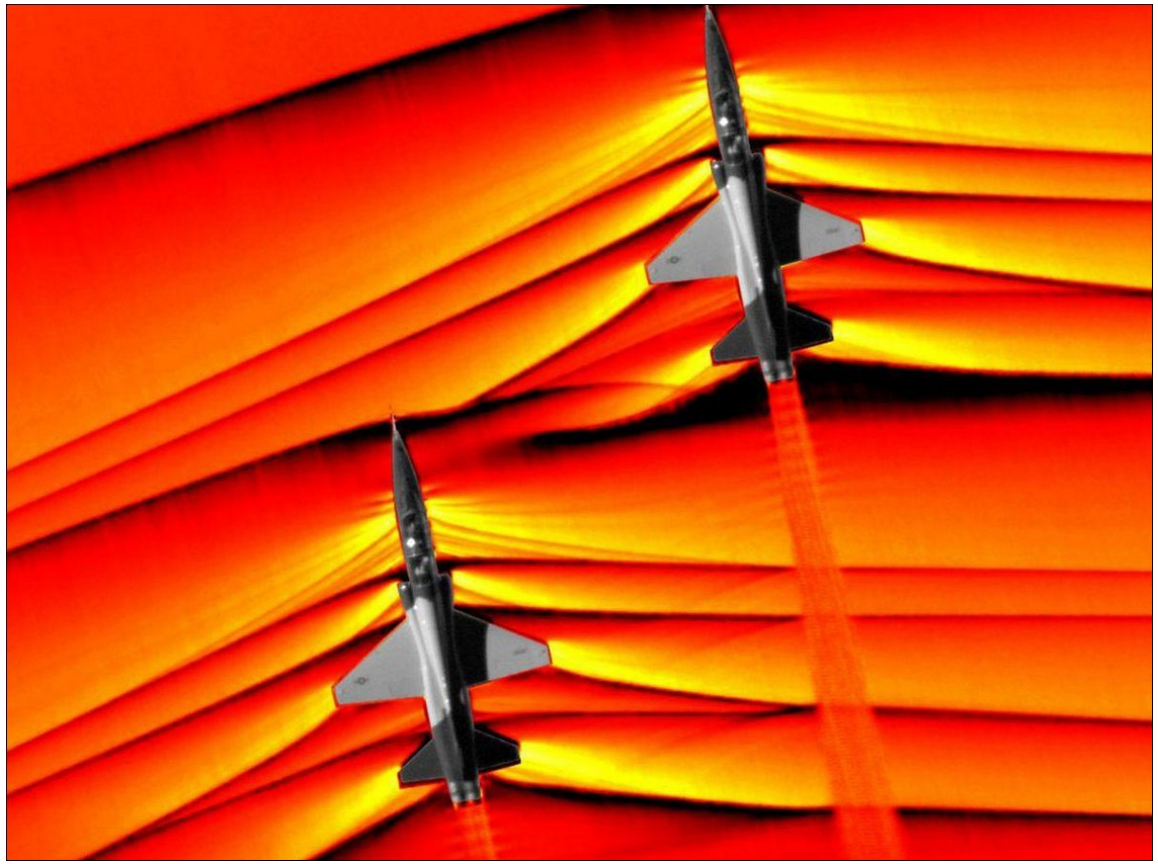
PHYSICS × NUMERICS × LEARNING



Capturing sharp fronts without spurious oscillations

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0$$

Shock capturing balances accuracy and stability



Schlieren imaging reveals abrupt changes in compressible flow.

The governing conservation law

$$\partial_t u + \partial_x f(u) = 0$$

$f(u) = au$ or $f(u) = \frac{u^2}{2}$

Smooth regions

Resolve the solution with high-order accuracy.

Discontinuities

Suppress nonphysical oscillations near shocks.

Euler flows

Maintain positive density and pressure.

> The learned component must work within the structure of a conservative numerical solver.

Local interface fluxes drive a conservative solver

01 Finite-volume discretization

$$\bar{u}_i(t) = \frac{1}{\Delta x}\int_{I_i} u(x,t)\,dx$$

$$L_i(u) = -\frac{\hat{f}_{i+1/2} - \hat{f}_{i-1/2}}{\Delta x}$$

02 Lax–Friedrichs interface flux

$$f^{\pm}(u) = \tfrac{1}{2}[f(u) \pm \alpha u]$$

$$\hat{f}_{i+1/2} = \hat{f}^{+}_{i+1/2} + \hat{f}^{-}_{i+1/2}$$

03 SSPRK3 time integration

$$u^{(1)} = u^n + \Delta t\, L(u^n)$$

$$u^{(2)} = \tfrac{3}{4}u^n + \tfrac{1}{4}\left[u^{(1)} + \Delta t L(u^{(1)})\right]$$

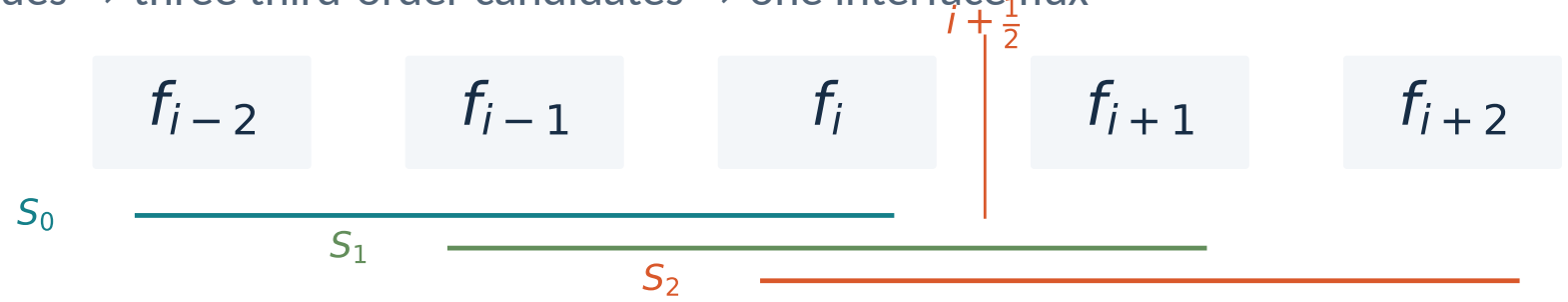
$$u^{n+1} = \tfrac{1}{3}u^n + \tfrac{2}{3}\left[u^{(2)} + \Delta t L(u^{(2)})\right]$$

The reconstruction is evaluated at every stage.

CFL time-step restriction $\Delta t \le \mathrm{CFL}\,\frac{\Delta x}{a_{\max}}$, $a_{\max} = \max_i(|v_i| + c_i)$ for Euler

WENO and TENO combine three local reconstructions

Five stencil values → three third-order candidates → one interface flux



ENO selects

Choose the smoothest candidate.

Select using divided differences.

$$\hat{f}_{i+1/2} = q_{k^*}$$

WENO weights

Reduce troubled candidates continuously.

$$\alpha_k = \frac{d_k}{(\varepsilon + \beta_k)^2}$$

$$\omega_k = \alpha_k / \|\alpha\|_1$$

TENO discards

Reject troubled candidates, then reweight.

$$\delta_k = \mathbf{1}[\chi_k \geq C_T]$$

$$\omega_k = \frac{d_k \delta_k}{d \cdot \delta}$$

$$\hat{f}_{i+1/2} = \omega_0 q_0 + \omega_1 q_1 + \omega_2 q_2, \qquad d = (0.1, 0.6, 0.3)$$

β: smoothness χ: TENO sensor

The network predicts which candidates to trust

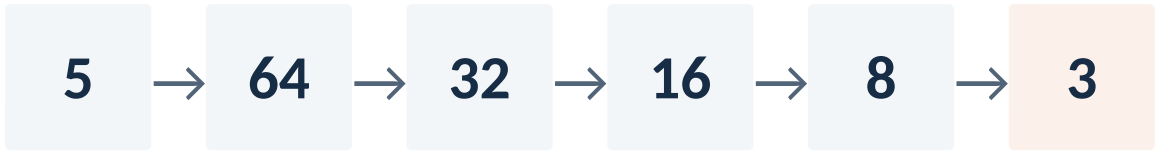
A local five-point stencil feeds a classifier with three troubled-candidate probabilities.

Normalize the input

$$x'_j = \frac{x_j - \min(x)}{\max(x) - \min(x)}$$

Constant stencils use a guarded denominator.

FEED-FORWARD CLASSIFIER



5 inputs — ReLU hidden layers — Sigmoid

$$(p_0, p_1, p_2) = \mathrm{NN}_\theta(x')$$

Training: continuous keep weights

$$s_k = 1 - p_k$$

Optional hard inference: threshold decisions

$$s_k = \mathbf{1}[p_k < \vartheta_k]$$

> The network changes candidate decisions; the solver still determines fluxes and time evolution.

Physics training evaluates the evolved solution

01 Imitation learning

Learn the classical TENO5 keep/discard labels.

$$p = \mathrm{NN}_\theta(x')$$

$$\mathcal{L}_{\mathrm{imit}} = \mathrm{BCE}(p, y_{\mathrm{TENO5}})$$

Useful initialization; the teacher supplies no direct signal for lower physical solution error.

02 Unrolled physics training

Differentiate through K complete SSPRK3 steps.

$$U_\theta^{n+K} = \mathcal{S}_\theta^K(U^n)$$

$$\mathcal{L}_{\mathrm{MSE}} = \mathrm{mean}\left[(U_\theta^{n+K} - U_{\mathrm{ref}}^{n+K})^2\right]$$

The gradient crosses reconstruction, fluxes and every Runge–Kutta substage.

Combined training objective

$$\mathcal{L} = \lambda_{\mathrm{MSE}}\mathcal{L}_{\mathrm{MSE}} + \lambda_{\mathrm{TV}}\mathcal{L}_{\mathrm{TV}} + \lambda_{\mathrm{BVD}}\mathcal{L}_{\mathrm{BVD}} + \lambda_{\mathrm{Reg}}\mathcal{L}_{\mathrm{Reg}} + \lambda_{+}\mathcal{L}_{+}$$

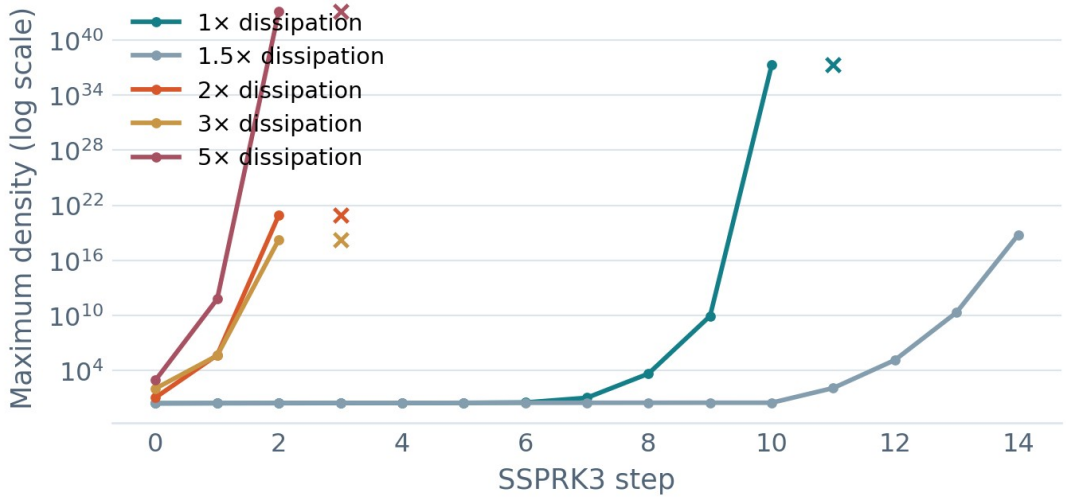
> Pretrain by imitation, normalize loss terms, then fine-tune against a physical reference.

Hybrid reconstruction supports stable Euler training

$$\Phi_{\mathrm{mix}} = (1-\lambda)\Phi_{\mathrm{TENO5}} + \lambda\Phi_{\mathrm{NN}}, \qquad 0 \le \lambda \le 1$$

A dedicated sigmoid head starts near TENO5: σ(−3) ≈ 0.047.

Observed Euler training instability



× marks a nonfinite state. Extra dissipation alone did not stabilize training.

Physical admissibility

$$\rho > 0, \qquad p = (\gamma - 1)\left(E - \frac{m^2}{2\rho}\right) > 0$$

m = ρv; γ = 1.4 for the project benchmarks.

Guard every substage

Measure positivity violations before repair.

Control optimization

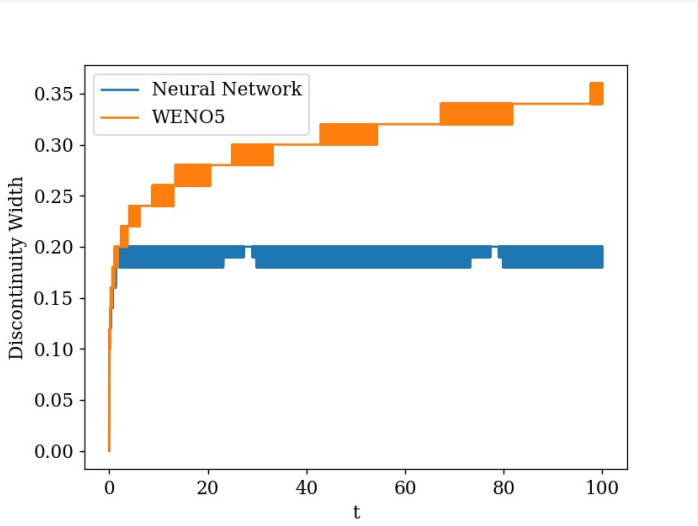
Use rollout curriculum, anchoring and clipping.

Validate the final solver

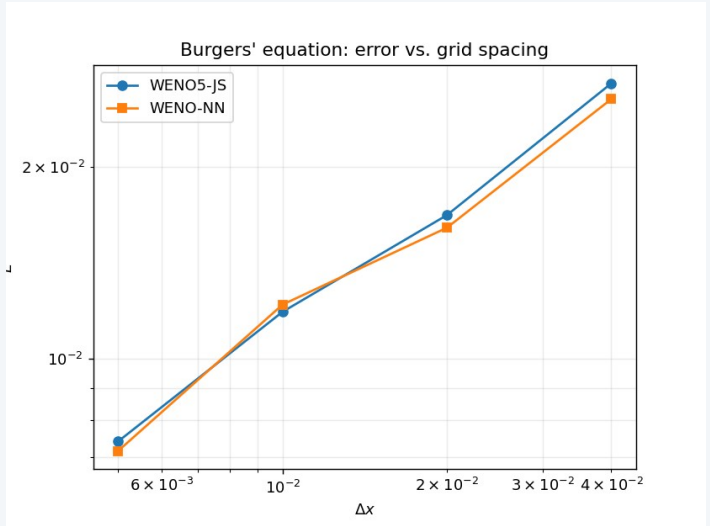
Mixing alone is not a positivity or TVD proof.

Final scalar benchmarks improve on TENO5

Earlier WENO-NN experiments



Sharper advected discontinuities



Competitive Burgers convergence

$$L_2 = \frac{\|u - u_{\text{ref}}\|_2}{\sqrt{N}}$$

Advection: $9.8960\times10^{-2} \rightarrow 9.5521\times10^{-2}$ | Burgers: $8.1670\times10^{-4} \rightarrow 6.6493\times10^{-4}$

Final NN-TENO5

Dedicated mixing head · Phase 19

3.5%

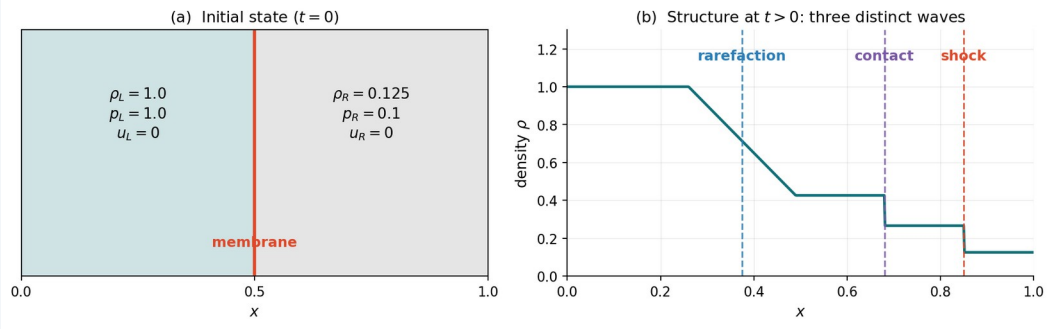
lower L_2 error on advection

18.6%

lower L_2 error on Burgers

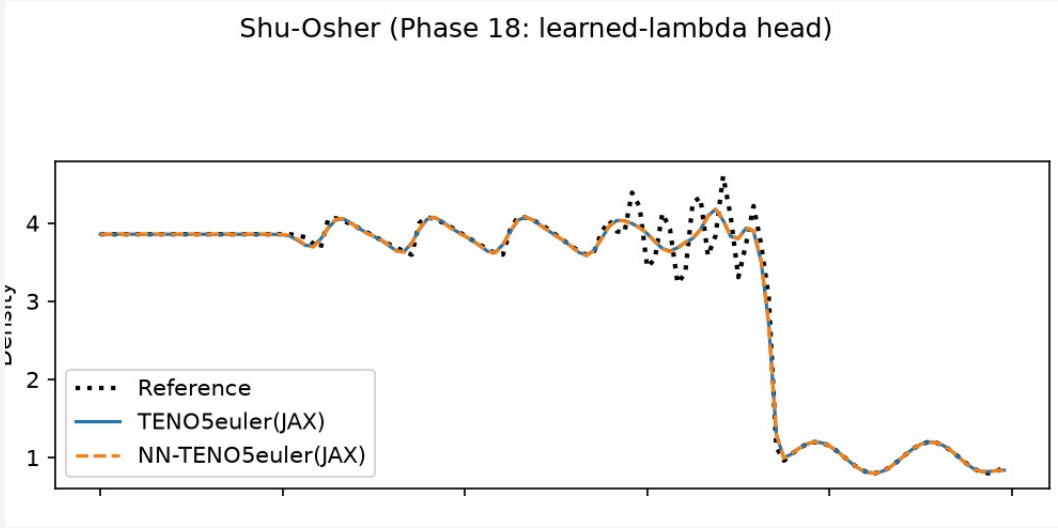
Euler gains depend on the benchmark

Sod shock tube



Rarefaction, contact discontinuity and shock

Shu–Osher shock–entropy interaction



+0.35% error

Near parity; TENO5 remains slightly better.

−1.84% error

A modest gain with the learned mixing head.

Metric: mean L_1 over (ρ, ρv, E). 120 cells, CFL 0.4. Sod: exact reference; Shu–Osher: 320-cell TENO5 reference.

Learning adds value inside a robust numerical solver

Learn the reconstruction decision.
Keep the numerical and physical structure visible.

What I built

WENO/TENO reconstruction framework

Neural stencil-decision models

Differentiable CFD training in JAX

What the results show

Scalar gains: 3.5% and 18.6% in L_2

Shu–Osher: 1.84% lower mean L_1

Sod: near parity, 0.35% higher error

What comes next

Improve the Euler reconstruction model

Validate across grids and flow cases

Extend the framework toward 2D CFD

Discussion

The physics loss balances accuracy and regularity

Accuracy

Mean square error against the evolved reference.

$$\mathcal{L}_{\mathrm{MSE}} = \mathrm{mean}\left[(U_\theta^{n+K} - U_{\mathrm{ref}}^{n+K})^2\right]$$

Total variation

Penalize extra variation; normalize by cell count.

$$\mathcal{L}_{\mathrm{TV}} = \frac{[\,TV(U^{n+K}) - TV(U^n)\,]_+}{N-1}$$

Boundary variation

Compare neighbors at the same interface.

$$BV_{i+1/2} = |\Phi_i(x_{i+1/2}) - \Phi_{i+1}(x_{i+1/2})|$$

$$\mathcal{L}_{\mathrm{BVD}} = \mathrm{mean}[\,BV_{\mathrm{NN}} - BV_{\mathrm{TENO5}}\,]_+$$

Anchor regularization

Limit drift from the stable pretrained parameters.

$$\mathcal{L}_{\mathrm{Reg}} = \frac{\|\theta - \theta_{\mathrm{anchor}}\|_2^2}{P}$$

$[z]_+ = \max(z, 0)$; $TV(U) = \Sigma_i\, |U_{i+1} - U_i|$; P = parameter count. Euler positivity is measured before state repair.

The learned decision sits inside every solver step

Forward numerical path

State + local stencil → Scalar / Euler setup → **WENO / TENO or NN selector** → Interface flux → SSPRK3 step → K-step rollout

Learned component

Euler: reconstruct characteristic fluxes

$$W = LU, \qquad G^{\pm} = \tfrac{1}{2} L[F(U) \pm \alpha U]$$

$$\hat{F}_{i+1/2} = R(\hat{G}^{+}_{i+1/2} + \hat{G}^{-}_{i+1/2}), \quad L = R^{-1}$$

Training: backpropagate physical error

$$\theta \leftarrow \mathrm{Adam}(\theta, \nabla_{\theta} \mathcal{L})$$

Reference → loss → gradient through every step → update the neural reconstruction parameters.

Scalar implementation: reconstruct states before Lax–Friedrichs flux evaluation. Euler: split characteristic fluxes before reconstruction.

Fifth-order reconstruction: candidates and TENO sensor

Three third-order candidate polynomials

$$q_0 = \frac{1}{3} f_{i-2} - \frac{7}{6} f_{i-1} + \frac{11}{6} f_i$$

$$q_1 = -\frac{1}{6} f_{i-1} + \frac{5}{6} f_i + \frac{1}{3} f_{i+1}$$

$$q_2 = \frac{1}{3} f_i + \frac{5}{6} f_{i+1} - \frac{1}{6} f_{i+2}$$

Optimal linear weights: d = (0.1, 0.6, 0.3).

TENO smoothness and scale separation

$$\tau_5 = |\beta_0 - \beta_2|$$

$$\gamma_k = \left(1 + \frac{\tau_5}{\beta_k + \varepsilon}\right)^6$$

$$\chi_k = \gamma_k / \|\gamma\|_1, \qquad \delta_k = \mathbf{1}[\chi_k \geq C_T]$$

$$\varepsilon = 10^{-40}, \qquad C_T = 10^{-5}$$

β_k are the Jiang–Shu smoothness indicators. NN-TENO5 learns candidate decisions with the same polynomials.
If every candidate is rejected, return the center value; otherwise normalize the retained weights.